

CHL5233: Quarto in-class assignment

Afra Alomgir

For this in-class assignment, you will work in a group to create your **individual** Quarto document. Please consult your classmates and [Tutorial: Authoring](#) webpage by [Quarto](#). You can also use the Internet including ChatGPT. The TA and I are also available for any questions.

When you are finished, render the document to PDF. Please carefully **proof read** and make edits as needed. **Commit** and **push** the edits to this file and the new PDF file to your **new repository**.

Create a new repository

- Under your GitHub account, create a **new** repository called `armed_conflict`
 - Throughout the course, we will use this repository to **reproduce** and **replicate** the analysis in the paper
- Create **two** folders
 - `data`
 - `reports`
- Save `countries.txt` and `regions.txt` into `data/raw/` subfolder
- Save `quarto_inclass.qmd` into `reports/` folder
- Eventually, your repository will have a very organized structure like this:

Example of data science organization

```
armed_conflict/  
  README.md  
  data/  
    raw/  
    processed/  
  scripts/  
    01_import.R  
    02_clean.R
```

```
03_analysis.R
outputs/
  figures/
  tables/
reports/
  analysis.qmd
```

Edit this file to create a PDF document using Quarto

The R code chunk below includes the R packages required to knit this document. This is often called a **setup** code. We want the code to run, but we don't want the final document to show the code or results. Edit the code chunk so that the code will run but the code and results (including all warnings and messages) are hidden from the final PDF output.

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
Warning: package 'readr' was built under R version 4.5.2
```

```
Warning: package 'dplyr' was built under R version 4.5.2
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.1.6
v forcats    1.0.1      v stringr    1.5.2
v ggplot2     4.0.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.1.0
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become er
```

```
Warning: package 'knitr' was built under R version 4.5.2
```

```
Attaching package: 'kableExtra'
```

```
The following object is masked from 'package:dplyr':
```

```
group_rows
```

For this exercise, we will use the following two data sets.

1. `countries.txt`: A data set that contains a list of countries used in the paper “Implications of armed conflict for maternal and child health: A regression analysis of data from 181 countries for 2000–2019” and their three-digit ISO (International Organization for Standardization) code.
2. `regions.txt`: A data set that contains a list of all major countries in the world and their
 - a. ISO code (`country_code`)
 - b. region (`region`)
 - c. sub-region (`sub_region`)
 - d. numerical region code (`region_code`)
 - e. numerical sub-region code (`sub_region_code`)

The **final goal** is to display a **bar chart** that depicts how many countries are in each region.

Read in data and create an analytical data set

First, edit the code chunk below that reads in `countries.txt` to also display the number of rows and columns of the data.

```
countries <- read.table("/Users/afraa/Desktop/week2-stat-programming/armed_conflict_AA/data/raw/countries.txt")
view(countries)
glimpse(countries)
```

```
Rows: 186
Columns: 2
$ country_name <chr> "Afghanistan", "Albania", "Algeria", "Andorra", "Angola", "~
$ ISO          <chr> "AFG", "ALB", "DZA", "AND", "AGO", "ATG", "ARG", "ARM", "~
```

There are 186 countries in the “countries” file.

Second, read in `regions.txt` and also write a sentence that states how many countries are included in the data set.

```
regions <- read.table("/Users/afraa/Desktop/week2-stat-programming/armed_conflict_AA/data/raw/regions.txt")
view(regions)
glimpse(regions)
```

```
Rows: 249
Columns: 6
$ country_name    <chr> "Afghanistan", "\xfd\xfdland Islands", "Albania", "Alg~
$ country_code    <chr> "AFG", "ALA", "ALB", "DZA", "ASM", "AND", "AGO", "AIA"~
$ region          <chr> "Asia", "Europe", "Europe", "Africa", "Oceania", "Euro~
$ sub_region      <chr> "Southern Asia", "Northern Europe", "Southern Europe", "~
```

```
$ region_code      <int> 142, 150, 150, 2, 9, 150, 2, 19, NA, 19, 19, 142, 19, ~
$ sub_region_code  <int> 34, 154, 39, 15, 61, 39, 202, 419, NA, 419, 419, 145, ~
```

There are 249 countries in the “regions” file

Third, subset the `regions` data set to only include countries from the `countries` data set.

```
regionsXcountries <- regions %>% filter(country_code %in% countries$ISO)
view(regionsXcountries)
```

Tabulate countries by region and sub-region

Next, we want to find out how many countries are in a region. Use the `group_by()` and `summarise()` functions in the `dplyr` package to count the number of countries in each region. Then use the `kable()` function in the `knitr` package to create a simple table with the column headers, “Region” and “Count”.

```
region_count <- regionsXcountries %>%
  group_by(region) %>%
  summarise(count = n())

view(region_count)

kable(region_count, col.names = c("Regions", "Count"), caption = "Number of Countries in a Region")
```

Table 1: Number of Countries in a Region

Regions	Count
Africa	54
Americas	35
Asia	46
Europe	40
Oceania	11

Create a similar table that shows the number of countries in each **sub-region**.

```
subregion_count <- regionsXcountries %>%
  group_by(sub_region) %>%
  summarise(count = n())

view(subregion_count)

kable(subregion_count, col.names = c("Sub-Regions", "Count"), caption = "Number of Countries in a Sub-Region")
```

Table 2: Number of Countries in a Subregion

Sub-Regions	Count
Australia and New Zealand	2
Central Asia	5
Eastern Asia	5
Eastern Europe	10
Latin America and the Caribbean	33
Melanesia	4
Micronesia	3
Northern Africa	6
Northern America	2
Northern Europe	10
Polynesia	2
South-eastern Asia	11
Southern Asia	9
Southern Europe	13
Sub-Saharan Africa	48
Western Asia	16
Western Europe	7

Data visualization using ggplot

Here, we will use `geom_bar()` function within `ggplot()` to create a bar chart that displays the **number of countries** in each **sub-region**.

1. Color code each sub-region by **region**.
2. Create bars that run horizontally where sub-region names appear along the y-axis.
3. Order the bars such that the sub-region with the most number countries appear at the top and the sub-region with the least number of countries appear at the bottom.

```
region_subregion_count <- regionsXcountries %>%
  group_by(region, sub_region) %>%
  summarise(count = n())
```

``summarise()`` has regrouped the output.

i Summaries were computed grouped by region and sub_region.

i Output is grouped by region.

i Use ``summarise(.groups = "drop_last")`` to silence this message.

i Use ``summarise(.by = c(region, sub_region))`` for per-operation grouping (``?dplyr::dplyr_by``) instead.

```
view(region_subregion_count)

ggplot(region_subregion_count, aes(x = count, y = fct_reorder(sub_region, count), fill = region))
  labs(x = "Number of Countries", y = "Sub-region", title = "Count of Countries with Armed Confl")
  geom_bar(stat = "identity") +
  theme_classic() + theme(plot.title = element_text(hjust = 0.5)) +
  scale_fill_brewer()
```

